

DIGITAL FORENSIC INVESTIGATION REPORT

Morris Worm Training Exercise: Process, Port, and File-Timeline Reconstruction in an Isolated SEED Nano-Internet

Case Title	Morris Worm Training Exercise: Process, Port, and File-Timeline Reconstruction in an Isolated SEED Nano-Internet
Case Identifier	CIP-B105-CS4-C11/26/DFIT/17310
Analyst	EBENEZER ELIKEM AMANKWAH OFORI DZAM
Report Date	September 26, 2026
Report Status	DRAFT (Working Merged Analysis Report)

STUDENT DECLARATION

I hereby declare that this forensic investigation report, the accompanying master provenance calculations, evidence correlation matrices, and analytical conclusions are entirely my own original work.

All derived sensitive image evidence and network extraction artifacts generated during this assessment have been handled strictly inside the isolated, protected IC DFA laboratory environment in compliance with safety controls. I confirm that original source evidence files have remained unedited, and all temporary working derivatives will undergo secure, authorized deletion (shredding) following final evaluation in accordance with IC DFA retention policy.

Student Name: Ebenezer Elikem Amankwah Ofori Dzam Date: September 25, 2026

Signature: E. E. A. O. Dzam

TABLE OF CONTENT

Table of Contents	Error! Bookmark not defined.
1. Executive Summary.....	3
2. Authority, Scope and Investigative Questions	4
2.1 Scope	4
2.2 Environment Topology.....	4
2.3 Investigative Questions	4
3. Evidence Inventory and Integrity Verification.....	5
4. Tools, Versions and Methodology.....	6
5. Detailed Findings and Evidential Interpretation.....	6
5.1 Historical Morris Worm Vectors vs. the Simplified Training Implementation.....	6
Mechanism reproduced	7
5.2 Pre-Infection Baseline	7
5.3 Process Forensics.....	7
Process-tree relationships (ps -eH --forest)	8
5.4 Listening-Port Forensics.....	8
5.5 File Hash and Timestamp Analysis	9
Attacker workstation (seed@VM: ~/Labsetup/worm)	9
Victim host 151/host_3 (~bof, container 1131478c9fe7).....	9
Victim host 152/host_2 — file-in-transfer artefact (Aug 4 continuation)	10
5.6 First Infection of the Victim	10
5.7 Propagation and Later Receipt/Overwrite	11
5.8 Clock Skew.....	11
5.9 Containment and Cleanup	11
6. Integrated Timeline	12
Additional events, container-relative time (clock skew not independently verified for these hosts)	12
7. Alternative Explanations, Limitations and Confidence Assessment	13
7.1 Alternative Explanations Considered	13
7.2 Limitations.....	13
7.3 Confidence Assessment.....	13
8. Professional Conclusion	14
Appendix A — Case Activity Log	14
Appendix B — Evidence Register.....	15
Appendix C — Hashes and File Metadata Summary	16
Appendix D — Commands / Queries Reference.....	16
References	16

1. Executive Summary

This report documents a controlled, in-lab reproduction and post-incident investigation of the historical Morris Worm (1988) attack technique, using the SEED Labs "Morris Worm" training exercise. The exercise was executed entirely inside an isolated Docker-based nano-internet (three emulated subnets, AS151/AS152/AS153, each with five hosts and a router, interconnected through a core router) built on a SEED-Ubuntu20.04 virtual machine. The training implementation reproduces only one of the four historical Morris Worm vectors a stack buffer overflow in a finger-daemon-style network service using a simplified Python control script (worm.py) and a crafted exploit payload (badfile).

Evidence was collected across the attacker workstation and four emulated victim hosts (153/host_4, 152/host_2, 153/host_2, and 151/host_3) using process listings (ps, ps aux, ps -eH --forest), listening-port enumeration (netstat -tulpn), file hashing (md5sum), and file-timestamp analysis (ls -l, stat), together with an explicit timezone/clock-skew check. The evidence establishes, with high confidence, the following infection and propagation sequence:

- The attacker authored worm.py (finalised 2022-08-03 08:09:23 EDT, MD5 428bda4544f74853835d5b765c52fa1b) and its exploit payload badfile (created 2022-08-03 08:18:15 EDT, MD5 0c4d6aacff3583b80adaf6e842b37e8a) on the attacker workstation.
- Host 151/host_3 shows a badfile with a corrected timestamp of 2022-08-03 08:18 EDT matching the attacker's badfile creation time to the minute together with a 315 KB core dump two minutes earlier (08:16 EDT), consistent with this host being the first victim compromised via the buffer-overflow crash.
- The same host subsequently shows a full copy of worm.py at a corrected 09:36 EDT, and again at 09:48 EDT, consistent with the worm being received, executed, and later re-received/overwritten direct evidence of propagation and repeated re-infection.
- Process evidence on 153/host_4 and 152/host_2 independently confirms live python3 worm.py processes, injected shellcode processes (visible as a distinctive echo '(^_^) Shellcode' command), and the worm's self-perpetuation beacon (repeated ping -q -i 1.2.3.4 child processes).
- Port evidence on 153/host_2 shows five separate nc listeners stacked up on TCP/9999 (the worm's control/transfer channel) alongside the exploited service itself listening on TCP/9090 direct evidence that this single host was successfully re-exploited at least five times.
- A timezone/clock-skew check on 151/host_2 shows the container clock is accurate in UTC but reads one hour behind true local time if interpreted as EST rather than EDT (the US Daylight-Saving zone in effect in August); a documented +1 hour correction is applied throughout this report's timeline wherever a source screenshot used TZ="EST".

The exercise was contained and the nano-internet and visualisation server shut down at the conclusion of evidence collection, consistent with the case package's authorised-scope and cleanup requirements. Section 7 sets out this report's confidence level, the alternative explanations considered, and its limitations — principally that clock-skew was verified explicitly on only one of the five hosts examined, and that no pre-infection baseline process/port capture was present in the supplied evidence.

2. Authority, Scope and Investigative Questions

Authorisation: This exercise was performed only within the isolated SEED Docker nano-internet described below, per the case package's "Authorisation and Safety" instruction. No worm code was deployed or modified for external systems, persistence, broader propagation, or real-world targeting, and the lab was stopped and cleaned immediately after evidence collection (Section 5.10).

2.1 Scope

The scope of this report is limited to artefacts collected from the attacker workstation and the emulated victim containers inside the nano-internet during a single supplied training run of the SEED Morris Worm lab. No physical hardware, external network, or production system was examined or affected.

2.2 Environment Topology

The nano-internet consists of three emulated autonomous systems, each a /24 subnet of five hosts (host_0–host_4) and one router (router0), interconnected through a core router (100/ix100, "NYC-100", 10.100.0.100/24):

Subnet (AS)	Network	Hosts	Router
AS151	10.151.0.0/24	host_0 (.71) – host_4 (.75)	router0 (.254)
AS152	10.152.0.0/24	host_0 (.71) – host_4 (.75)	router0 (.254)
AS153	10.153.0.0/24	host_0 (.71) – host_4 (.75)	router0 (.254)

Each container is built from a per-host Dockerfile inheriting a shared morris-worm-base image, orchestrated by docker-compose.yml. A visualisation server (map.html, served on localhost:8080) displays live topology and flashes any host observed sending ICMP traffic to 1.2.3.4 the training worm's deliberate "I am infected" beacon.

2.3 Investigative Questions

The following questions, drawn directly from the case package's Required Findings, define the scope of Section 5:

#	Question	Addressed in
1	Which historical Morris Worm attack vectors are described by the source?	5.1
2	What is different about the simplified SEED training implementation?	5.1
3	What evidence establishes the pre-infection baseline?	5.2
4	Which process and listening port support the training transfer/control stage?	5.3, 5.4
5	What process-tree relationships are visible?	5.3
6	What do worm.py and badfile hashes/timestamps show?	5.5
7	Which event best supports first infection of the victim?	5.6
8	What evidence supports propagation or later receipt/overwrite?	5.7
9	How does clock skew affect the sequence?	5.8, 6

#	Question	Addressed in
10	What evidence confirms containment and cleanup?	5.9

3. Evidence Inventory and Integrity Verification

Evidence in this report originates from two case-supplied slide decks documenting a live walkthrough of the exercise: Morris_Attack.pptx (lab build and worm execution) and Morris_Forensics.pptx (post-incident evidence collection). Embedded terminal screenshots within these decks are the primary evidentiary source; Appendix D indexes every screenshot used.

Evidence item	Location / host	Type	Integrity check
worm.py (attacker original)	seed@VM: ~/Labsetup/worm	Python script, 3695 bytes	MD5 428bda4544f74853835d5b765c52fa1b
badfile (attacker original)	seed@VM: ~/Labsetup/worm	Binary exploit payload, 500 bytes	MD5 0c4d6aacff3583b80adaf6e842b37e8a
worm.py (victim copy)	151/host_3: ~/bof	Python script, 3695 bytes (2 captures)	Size matches attacker original; MD5 not re-verified on this host in the supplied evidence
badfile (victim copy)	151/host_3: ~/bof	Binary exploit payload, 500 bytes	Size matches attacker original; MD5 not re-verified on this host in the supplied evidence
core	151/host_3: ~/bof	Process core dump, 315,392 bytes	Not hashed in the supplied evidence; timestamp-only artefact
server / stack	151/host_3: ~/bof	Baseline binaries, 17,768 / 709,188 bytes, dated Jan 21 2022	Unchanged from pre-exercise baseline (no Aug 2022 modification observed)
ps aux output	153/host_4, 152/host_2	Process listing (screenshot)	Captured directly from live containers; no independent hash applicable
netstat -tulpn output	153/host_2	Listening-port listing (screenshot)	Captured directly from live container; no independent hash applicable
Topology / map.html capture	Visualisation server	Network diagram (screenshot)	Captured directly from the running emulator

Limitation: Victim-side copies of worm.py and badfile are corroborated by matching file size and, for badfile on 151/host_3, a timestamp that aligns to the minute with the attacker's original. No screenshot in the supplied evidence shows an md5sum re-run against a victim-host copy of either file, so byte-for-byte identity between

Evidence item	Location / host	Type	Integrity check
the attacker original and any victim copy is inferred from size and timing, not independently hash-verified. This is recorded as a limitation in Section 7, not presented as a confirmed hash match.			

4. Tools, Versions and Methodology

Tool / component	Purpose
SEED-Ubuntu20.04 (VirtualBox VM)	Attacker/analyst workstation; hosts the Docker engine and lab files
Docker + docker-compose	Builds and orchestrates the nano-internet containers from Dockerfile / docker-compose.yml
Internet Emulator visualisation server (map.html, port 8080)	Live topology view; flashes hosts observed pinging 1.2.3.4 (the worm's infection beacon)
worm.py / badfile	Training exploit: worm.py builds badfile (a crafted buffer-overflow payload with embedded shellcode), delivers it to a target's vulnerable service, and propagates a copy of itself once control is gained
ps, ps aux, ps -ef, ps -eH --forest	Process and process-tree enumeration on victim/attacker hosts
netstat -tulpn	Listening TCP/UDP port enumeration, mapped to owning PID/process
ls -l / ll, stat	File size, permission and mtime/ctime/ctime (MACB) timestamp analysis
md5sum	Cryptographic hashing of worm.py and badfile on the attacker host
date, TZ environment variable	Wall-clock and timezone verification, used to detect and correct clock skew
tcpdump / seedemu_sniffer	Background packet capture running inside each container (present in ps aux output; not separately analysed in this report)
Note: Every finding in Section 5 is sourced to a specific screenshot (Appendix D) and a specific command shown in that screenshot. Where a command or its output is not visible in the supplied evidence — for example, no packet-capture (tcpdump/pcap) analysis was supplied, and no explicit pre-infection ps/netstat baseline capture was supplied — this report says so explicitly rather than inferring the missing artefact.	

5. Detailed Findings and Evidential Interpretation

5.1 Historical Morris Worm Vectors vs. the Simplified Training Implementation

The source material (Morris_Attack.pptx) documents four historical Morris Worm (1988) attack vectors used by its author, Robert Tappan Morris, then a Cornell graduate student:

- A bug (buffer-overflow-style error) in sendmail.

- A stack buffer overflow in the finger daemon, reached via an old, unbounded gets() call reading a user query into a 512-byte stack buffer.
- Abuse of the "trusted hosts" (rhosts/setuid-style) feature, allowing privileged access on one host to be reused on another without a password.
- A password-guessing routine trying rapid combinations against authorised accounts.

The SEED training implementation reproduces only the second vector — the finger-daemon-style stack buffer overflow — as a deliberately simplified Python exploit chain. The remaining three historical vectors (sendmail, trusted hosts, password guessing) are not present anywhere in the supplied evidence and were not exercised in this lab.

Mechanism reproduced

- worm.py crafts badfile, an exploit payload containing shellcode and follow-on commands (injection).
- badfile is delivered to a target's vulnerable service via the buffer overflow.
- The shellcode inside badfile is executed on the victim, starting an nc listener that waits for and executes an incoming copy of worm.py (control).
- The attacker sends worm.py to the now-listening victim over that channel (spreading).
- worm.py on the victim runs in an infinite loop, scanning the training IP range (10.X.0.Y, X=151–153, Y=71–75) for the next target (self-perpetuation).

Note: The historical Morris Worm additionally contained self-avoidance logic a check for an existing infection that Morris deliberately weakened to a roughly 1-in-7 chance of reinfecting anyway, a decision widely cited as the cause of the original worm's runaway resource-exhaustion effect on the 1988 Internet. The training worm.py similarly includes a stated existence-of-badfile self-check intended to prevent re-infection; however, this report's own port and process evidence (Sections 5.3–5.4) shows repeated successful re-exploitation of the same hosts, indicating that self-check did not prevent repeat infection in this run, mirroring for different underlying reasons the historical worm's repeat-infection behaviour. Both are non-destructive by design; neither the 1988 worm nor this training reproduction was intended to corrupt or exfiltrate data.

5.2 Pre-Infection Baseline

The evidence establishes the nano-internet's topology and addressing scheme as a pre-infection baseline: three /24 subnets (AS151–AS153), each with five hosts and a router, mapped live via the visualisation server before any attack traffic was sent (Section 2.2). On 151/host_3, the vulnerable service binary (server) and its associated stack file carry unchanged timestamps of Jan 21 2022, pre-dating the 2022-08-03 exercise by over six months — evidencing that these files were part of the container's built image and were not modified during the exercise itself.

Limitation: No screenshot in the supplied evidence shows a process listing (ps aux) or listening-port capture (netstat) taken from a victim host before the attack was launched. The pre-infection baseline in this report is therefore limited to the network topology and the unmodified-since-build timestamps on the server/stack binaries; it does not include a "before" process or port snapshot to compare directly against the "after" captures in Sections 5.3–5.4. This is recorded as a limitation in Section 7.

5.3 Process Forensics

ps aux was captured on two infected hosts and shows consistent, independent confirmation of the worm mechanism described in 5.1:

Host	Container ID	PID	START	Command	Interpretation
153/host_4	e8a4a2a503be4	70	01:40	python3 worm.py	Worm process running under root
153/host_4	e8a4a2a503be4	67	01:40	/bin/bash -c echo '(^_^) Shellcod[e]'	Injected shellcode payload executed — direct proof of successful exploitation
153/host_4	e8a4a2a503be4	73–424+	01:40–01:48	ping -q -i 1.2.3.4 (repeated)	Worm's infection beacon, visualised on the topology map
152/host_2	feb18fcd8b31	68	02:15	python3 worm.py	Worm process running under root
152/host_2	feb18fcd8b31	66	02:14	/bin/bash -c echo '(^_^) Shellcode is running (...'	Injected shellcode payload executed
152/host_2	feb18fcd8b31	249, 261	02:18–02:19	echo Shellcode / nc -lnv 9999 (2nd pair)	A second, later shellcode+listener pair — evidence of repeat exploitation of the same host

Interpretation: on both hosts, the shellcode-echo process and the nc listener process are timestamped within one minute of each other and immediately precede the corresponding python3 worm.py process, consistent with the documented mechanism (injection → control → spread). The repeated ping -q -i 1.2.3.4 children are the worm's deliberate self-identification beacon, used by the visualisation server to flash infected hosts on the topology map; their volume and repetition (dozens of short-lived child processes) is consistent with the worm looping indefinitely as designed.

Process-tree relationships (ps -eH --forest)

The forest-style listing confirms standard Linux kernel-thread ancestry (kthreadd as parent of rcu_gp, rcu_par_gp, and related kernel worker threads) for baseline OS processes. Applied to the worm activity above, the shellcode process (echo) and the nc listener are siblings spawned from the exploited service's crashed/hijacked execution context, and python3 worm.py is the child ultimately launched once nc receives and executes the transferred worm.py consistent with the 5-step mechanism in Section 5.1 rather than an independently, manually started process.

5.4 Listening-Port Forensics

netstat -tulpn on 153/host_2 (container 2dc1262d7ddd) enumerates all listening TCP/UDP endpoints on that host:

Local address	State	PID/Program	Interpretation
0.0.0.0:9999 (x5)	LISTEN	505/nc, 490/nc, 364/nc, 363/nc, 338/nc	Five separate nc control-channel listeners, one per successful exploitation of this host direct evidence this host was compromised at least five times
0.0.0.0:9090	LISTEN	58/./server	The vulnerable finger-daemon-style service itself the exploited entry point
127.0.0.1:35057 / 35700	LISTEN	—	Local-only loopback services, unrelated to the worm channel

Interpretation: TCP/9999 is the worm's fixed control/transfer port (matching the shellcode's `nc -lnv 9999` command observed in Section 5.3 and the training configuration in `Morris_Attack.pptx`); TCP/9090 is the fixed port of the exploited service itself. The presence of five concurrently listening, distinctly-PID'd `nc` processes on the same port is the clearest single artefact in this examination of repeated, successful re-exploitation of one host, consistent with the worm's continuous scan-and-attack loop (Section 5.1) and the repeat-infection pattern noted for the historical worm.

5.5 File Hash and Timestamp Analysis

Attacker workstation (seed@VM: ~/Labsetup/worm)

File	Size	Perms	MD5	Access (atime)	Modify (mtime)	Change (ctime)
worm.py	3695	rw-rw-r-x	428bda45...44f7...4853835d5b765c52fa1b	2022-08-03 08:10:18.746 -0400	2022-08-03 08:09:23.459 -0400	2022-08-03 08:09:23.463 -0400
badfile	500	rw-rw-r--	0c4d6aac...ff3583b80adaf6e842b37e8a	2022-08-03 08:18:15.312 -0400	2022-08-03 08:18:15.308 -0400	2022-08-03 08:18:15.308 -0400

- Both files share inode device 805h/2053d; worm.py is inode 1575830 and badfile is inode 1575831 sequential inode allocation consistent with badfile being created immediately after worm.py was finalised, by the same process, on the same filesystem.
- The attacker workstation's date command independently reads Wed 03 Aug 2022 08:33:28 AM EDT, confirming the workstation's own clock is already in EDT (no correction needed for attacker-side timestamps in this report).

Victim host 151/host_3 (~bof, container 1131478c9fe7)

All timestamps below were captured with TZ="EST" and require the +1 hour correction documented in Section 5.8 to align with the attacker workstation's native EDT timestamps; both the raw (EST, as captured) and corrected (EDT) values are given.

File	Size	Raw (EST, as captured)	Corrected (EDT, +1h)	Interpretation
core	315,392	2022-08-03 07:16	2022-08-03 08:16	Crash dump from the exploited service — direct evidence the buffer-overflow attempt crashed the target process

File	Size	Raw (EST, as captured)	Corrected (EDT, +1h)	Interpretation
badfile	500	2022-08-03 07:18	2022-08-03 08:18	Matches the attacker's badfile creation time (08:18:15 EDT) to the minute
server	17,768	Jan 21 2022 (unchanged)	Jan 21 2022 (unchanged)	Pre-exercise baseline binary, not modified during the incident
stack	709,188	Jan 21 2022 (unchanged)	Jan 21 2022 (unchanged)	Pre-exercise baseline binary, not modified during the incident
worm.py (1st capture)	3,695	2022-08-03 08:36	2022-08-03 09:36	First full receipt of worm.py on this host
worm.py (2nd capture)	3,695	2022-08-03 08:48	2022-08-03 09:48	worm.py overwritten again by a later re-infection cycle

Victim host 152/host_2 — file-in-transfer artefact (Aug 4 continuation)

A repeated `ls -l` polling capture on 152/host_2 (container feb18fcd8b31) on 2022-08-04 shows worm.py's size oscillating between 0 bytes and its full 3,695 bytes across successive polls (02:15–02:29), while badfile remains a constant 500 bytes throughout:

Poll time	badfile size	worm.py size	Interpretation
02:15	500	— (badfile just arrived)	New exploitation cycle begins; badfile freshly delivered
02:25 (x4 polls)	500	0	nc has opened/truncated worm.py for the incoming transfer but the copy is not yet complete — file caught mid-transfer
02:26	500	3,695	Transfer completes; full worm.py momentarily present on disk
02:29	500	0	A further re-infection cycle begins, truncating worm.py again

Interpretation: badfile's constant size and worm.py's oscillation between 0 and 3,695 bytes is direct evidence of nc-based file transfer being observed live, and of this host being repeatedly re-targeted, consistent with the five-listener finding on 153/host_2 (Section 5.4).

Note: badfile does not change size or content across any capture in the supplied evidence, while worm.py repeatedly changes (in size, during transfer, and in timestamp, across re-infection cycles). This asymmetry is expected: badfile is a static, pre-built exploit payload, whereas worm.py is retransmitted and re-executed fresh at every successful compromise.

5.6 First Infection of the Victim

The single strongest evidentiary link in this examination is the badfile timestamp match: the attacker's badfile was created at 2022-08-03 08:18:15.308 EDT, and 151/host_3's badfile carries a corrected timestamp of 2022-08-03 08:18 EDT matching to the minute (sub-minute precision is not available from the victim-side ls -l capture). Combined with the core dump on the same host timestamped two minutes earlier (08:16 EDT, consistent with the exploited service crashing during the buffer-overflow attempt that delivered badfile), this is assessed as the best-supported candidate for the first successful infection event captured in the evidence.

5.7 Propagation and Later Receipt/Overwrite

151/host_3's two worm.py captures (09:36 EDT and 09:48 EDT, corrected) are direct evidence of propagation continuing well after the initial 08:18 EDT compromise: the worm was received and executed roughly 78 minutes after first infection, and again 12 minutes after that. Independently, 152/host_2's Aug-4 polling capture and 153/host_2's five stacked nc listeners (Section 5.4) both show the same host being re-compromised multiple times, which is the clearest port/process-level evidence of propagation and repeat receipt across the examined hosts.

5.8 Clock Skew

A timezone check was captured on 151/host_2 (container 68e9265eba02), a different container in the same subnet as 151/host_3:

Command	Result
TZ="EST" date	Wed Aug 3 06:56:50 EST 2022
date (container default)	Wed Aug 3 11:56:53 UTC 2022
Host physical clock (taskbar)	7:57 AM, 8/3/2022

11:56:53 UTC converts to 07:56:53 under EDT (UTC-4), which matches the host's physical clock (7:57 AM) to within normal rounding/latency confirming the container's underlying clock is accurate in absolute (UTC) terms. However, 11:56:53 UTC converts to 06:56:53 under EST (UTC-5), matching the TZ="EST" reading (06:56:50) almost exactly. Because the exercise was run in early August, US clocks observe Eastern Daylight Time (EDT, UTC-4), not Eastern Standard Time (EST, UTC-5); any container timestamp read out using TZ="EST" is therefore exactly one hour behind true local wall-clock time.

Limitation: This +1 hour correction has been applied throughout this report wherever a source screenshot explicitly used TZ="EST" (151/host_3's file listings in Section 5.5). The attacker workstation's own date output already read EDT directly and required no correction. The clock-skew check itself was captured on only one host (151/host_2); it was not independently re-verified on 153/host_4, 152/host_2, or 153/host_2, so the START times reported for those hosts in Sections 5.3-5.4 are presented as raw, uncorrected container-relative times and are not converted to EDT in this report — see Section 7.

5.9 Containment and Cleanup

Per the training material (Morris_Attack.pptx), containment consisted of shutting down the nano-internet and visualisation server using dcdownd in both the internet-nano and map directories once evidence collection was complete, and the case package's Authorisation and Safety instruction required the lab to be stopped and cleaned immediately after evidence collection. No screenshot in the supplied evidence captures the dcdownd command's output or a subsequent verification (e.g. docker ps showing no lab containers running); the shutdown step is documented in the walkthrough material as the prescribed procedure, but is not independently evidenced with an executed-and-verified screenshot in this examination's evidence set.

Limitation: Containment procedure (stopping processes and running dcdownd) is documented as required lab procedure but is not evidenced by a post-shutdown verification screenshot in the supplied materials. This is recorded as a limitation rather than assumed to have been completed and verified.

6. Integrated Timeline

All times below are corrected to EDT (America/New_York, UTC−4) where a source screenshot supplied a raw EST or UTC reading and the +1 hour / +4 hour conversion documented in Section 5.8 has been applied; the source column shows exactly what was captured. Container-relative START times from ps aux on 153/host_4 and 152/host_2, and the netstat capture on 153/host_2, could not be converted with confidence (Section 5.8) and are listed separately, uncorrected, at the end of the table.

Date / time	Event	Source artefact	Confidence
2022-08-03 08:09:23 EDT	worm.py last modified (finalised) on attacker workstation	stat worm.py (mtime)	Direct
2022-08-03 08:10:18 EDT	worm.py accessed (read) on attacker workstation	stat worm.py (atime)	Direct
2022-08-03 08:16 EDT (corrected)	Exploited service crashes on 151/host_3 (core dump written)	ls -l bof (TZ=EST, +1h)	Direct
2022-08-03 08:18:15 EDT	badfile created on attacker workstation	stat badfile (mtime/ctime)	Direct
2022-08-03 08:18 EDT (corrected)	badfile received on 151/host_3 — assessed as first infection	ls -l bof (TZ=EST, +1h)	Direct — matches attacker badfile time to the minute
2022-08-03 09:36 EDT (corrected)	worm.py received/executed on 151/host_3	ls -l bof (TZ=EST, +1h)	Direct
2022-08-03 09:48 EDT (corrected)	worm.py received/executed again on 151/host_3 (overwrite)	ls -l bof (TZ=EST, +1h)	Direct
2022-08-04 02:15–02:29 (uncorrected, container-relative)	Repeated worm.py transfer-in-progress cycles observed on 152/host_2	ls -l bof (repeated polling)	Direct, timezone not independently verified for this host

Additional events, container-relative time (clock skew not independently verified for these hosts)

Host	Container-relative time	Event
153/host_4	01:40	python3 worm.py and shellcode-echo processes observed running

Host	Container-relative time	Event
153/host_4	01:40–01:48	Repeated ping -q -i 1.2.3.4 infection-beacon children observed
152/host_2	02:14–02:15	Shellcode-echo process and python3 worm.py observed running (first compromise)
152/host_2	02:18–02:19	Second shellcode-echo / nc listener pair observed (re-compromise)
153/host_2	not timestamped in capture	netstat -tulpn shows five stacked nc listeners on TCP/9999 (repeat compromise) and the exploited service on TCP/9090

7. Alternative Explanations, Limitations and Confidence Assessment

7.1 Alternative Explanations Considered

- Manual analyst interaction rather than autonomous worm behaviour: the repeated ls polling on 152/host_2 and the multiple shellcode/listener pairs on 152/host_2 and 153/host_2 could in principle reflect an analyst manually re-running the exploit for demonstration purposes rather than the worm autonomously re-scanning and re-infecting. The presence of the worm's own infinite-loop scan logic (documented in the training source material, Section 5.1) and the multiplicity of distinct PIDs make autonomous re-infection the better-supported explanation, but manual repetition during a training walkthrough cannot be fully excluded from process/file evidence alone.
- Container clock drift specific to each host: this report assumes the clock-skew behaviour verified on 151/host_2 (UTC-accurate, EST-mislabelled) generalises to other containers built from the same base image; if any individual container's clock had independently drifted, the uncorrected container-relative times in Section 6's second table could be off by more than the documented one hour. This is why those times are presented uncorrected rather than force-fitted to the EDT timeline.

7.2 Limitations

- No pre-infection ps/netstat baseline capture is present in the supplied evidence (Section 5.2); "before" and "after" process/port states cannot be directly diffed.
- No md5sum re-run against any victim-host copy of worm.py or badfile is present in the supplied evidence; victim-copy integrity rests on matching file size (and, for badfile on 151/host_3, a matching timestamp), not an independently confirmed hash match.
- The explicit TZ/clock-skew check was captured on only one host (151/host_2); its result has been applied to 151/host_3's TZ="EST" captures (same subnet, same base image) but not to 153/host_4, 152/host_2, or 153/host_2, whose times are reported uncorrected.
- No packet capture (pcap) analysis is included in the supplied evidence, despite tcpdump/seedemu_sniffer processes being visible in every ps aux capture; this report does not claim network-payload-level confirmation of the exploit or transfer beyond the process/port/file artefacts described.
- Containment (dcdownd) is documented as required procedure but not evidenced by a post-shutdown verification screenshot (Section 5.9).
- This is a training reproduction of one historical Morris Worm vector; findings about the training worm's behaviour should not be read as a complete technical account of the 1988 Morris Worm itself, which also exploited sendmail, trusted hosts, and password guessing — none of which were exercised in this lab.

7.3 Confidence Assessment

The core infection-and-propagation finding that the exercise reproduced a working buffer-overflow-based worm that compromised multiple hosts and repeatedly re-infected at least two of them — is assessed with high confidence, resting on convergent process, port, and file-timestamp evidence across four independent hosts. The precise first-infection timestamp (2022-08-03 08:18 EDT on 151/host_3) is assessed with high confidence based on the minute-level match to the attacker’s own badfile creation time. The exact container-relative timings for 153/host_4, 152/host_2, and 153/host_2 are assessed with moderate confidence only, pending independent clock-skew verification on those specific hosts.

8. Professional Conclusion

This examination confirms that the SEED Labs Morris Worm training exercise, as executed in the supplied isolated nano-internet, successfully reproduced the historical worm’s finger-daemon buffer-overflow vector in simplified form and exhibited the same qualitative behaviour as the 1988 original on a much smaller scale: a single crafted exploit payload led to process injection, remote control via a listening shell, self-propagation to further hosts, and notably repeated re-infection of already-compromised hosts, mirroring the resource-exhaustion pattern for which the original Morris Worm is best known. The evidence identifies 151/host_3 as the best-supported first-infection point (matching the attacker’s badfile creation time to the minute) and documents at least four hosts across all three subnets showing direct evidence of compromise. The training implementation is deliberately narrower than the historical attack, reproducing only one of Morris’s four original vectors and omitting sendmail, trusted-hosts abuse, and password guessing entirely.

The findings in this report are limited to what the supplied process, port, and file-timestamp evidence directly shows; no claim is made beyond that evidentiary basis, and the limitations in Section 7 particularly the absence of a pre-infection baseline capture and of victim-side hash verification should be read alongside every conclusion above.

Appendix A — Case Activity Log

Date/time (as captured)	Action	Tool/method	Source evidence	Output/observation
2022-08-03 08:09 EDT	Finalised worm.py on attacker workstation	Text editor + md5sum	seed@VM shell	worm.py hashed and timestamped
2022-08-03 08:18 EDT	Ran worm.py; badfile generated and delivered	python3 worm.py	seed@VM shell	badfile created; first host (151/host_3) compromised
2022-08-03 ≈09:36–09:48 EDT	Propagation continues to 151/host_3 (repeat receipt)	Worm self-propagation	151/host_3 shell (TZ=EST)	worm.py overwritten twice
(container-relative) 01:40–01:48	Process evidence collected	ps aux	153/host_4 shell	worm.py, shellcode, ping-beacon processes observed
(container-relative) 02:14–02:19	Process evidence collected	ps aux	152/host_2 shell	Two shellcode/listener cycles observed

Date/time (as captured)	Action	Tool/method	Source evidence	Output/observation
Not timestamped in capture	Port evidence collected	netstat -tulpn	153/host_2 shell	Five nc listeners on 9999; server on 9090
2022-08-04 02:15–02:29	File-transfer-in-progress evidence collected	Repeated ls -l bof polling	152/host_2 shell	worm.py size oscillating 0 ↔ 3,695 bytes
Not dated in capture	Containment	dcdownd (internet-nano, map)	Morris_Attack.pptx walkthrough	Documented procedure; not independently verified in evidence

Appendix B — Evidence Register

Evidence ID	Source / host	Type	Size / hash	Evidential purpose
EV-01	Attacker workstation	File (worm.py)	3695 B / MD5 428bda45...	Establishes original worm code and authoring timestamp
EV-02	Attacker workstation	File (badfile)	500 B / MD5 0c4d6aac...	Establishes original exploit payload and creation timestamp
EV-03	Attacker workstation	stat output	n/a	Sub-second MACB timestamps for worm.py and badfile
EV-04	151/host_3	ls -l output (x3)	n/a	Victim-side badfile/core/worm.py timestamps; clock-skew-affected
EV-05	151/host_2	date output	n/a	Establishes container clock skew (EST vs true EDT)
EV-06	153/host_4	ps aux output	n/a	Confirms worm.py process, shellcode execution, infection beacon
EV-07	152/host_2	ps aux output	n/a	Confirms second-host compromise and repeat exploitation
EV-08	153/host_2	netstat -tulpn output	n/a	Confirms 5x repeat compromise via stacked port-9999 listeners
EV-09	152/host_2	Repeated ls -l bof output	n/a	Demonstrates live file-transfer-in-progress behaviour

Evidence ID	Source / host	Type	Size / hash	Evidential purpose
EV-10	Visualisation server	Topology screenshot	n/a	Establishes nano-internet baseline topology and addressing

Appendix C — Hashes and File Metadata Summary

File	Location	Size (bytes)	MD5
worm.py	Attacker workstation	3695	428bda4544f74853835d5b765c52fa1b
badfile	Attacker workstation	500	0c4d6aacff3583b80adaf6e842b37e8a
worm.py	151/host_3	3695	Not independently hashed in supplied evidence (size-matched only)
badfile	151/host_3	500	Not independently hashed in supplied evidence (size-matched only)
core	151/host_3	315,392	Not hashed (crash-dump timestamp artefact only)

Appendix D — Commands / Queries Reference

Command	Run on	Purpose
ps / ps -e / ps aux / ps -ef	Attacker VM, victim containers	Basic and full-format process listings
ps -eH --forest	Attacker VM	Process-tree / parent-child visualisation
netstat -tulpn grep LISTEN	153/host_2	Listening TCP/UDP ports mapped to owning PID/process
ll / ls -l, ls -cl, ls -ul	Attacker VM, 151/host_3	mtime, ctime, atime timestamp inspection
stat <file>	Attacker VM	Full sub-second MACB timestamp and inode detail
md5sum <file>	Attacker VM	Cryptographic hash of worm.py / badfile
date / TZ="EST" date	151/host_2	Wall-clock and timezone-skew verification
nc -lnv 9999 > worm.py && python3 worm.py	Victim containers (via shellcode)	Worm control channel: receive and execute worm.py
dcdowndown	internet-nano/, map/ directories	Shut down the nano-internet and visualisation server (containment)

References

- CIP-B105-C4-Case_Senario.docx (case-supplied assessment brief, Module 4)
- Morris_Attack.pptx (case-supplied SEED lab build/execution walkthrough, 48 slides)
- Morris_Forensics.pptx (case-supplied post-incident evidence collection walkthrough, 27 slides)
- SEED Labs, "Morris Worm" lab manual,
https://seedsecuritylabs.org/Labs_20.04/Networking/Morris_Worm/
- U.S. v. Morris case summary, http://www.loundy.com/CASES/US_v_Morris2.html